

M03 - Advanced SQL + PostgreSQL / Database Thinking

16-lesson learner book | Beginner-first professional institute standard | B01-style teaching contract

Purpose

Control joins, multi-step SQL, windows, PostgreSQL concepts and query reasoning.

Course boundary

This is the independent Standalone Data Engineer course, not Four-Course C3. Complete the course in sequence; each new idea is still taught from first principles.

Option B institute identity | Charcoal #1F2937 | Teal #14B8A6 | Soft Blue #3B82F6

This book is your teacher

Use one lesson at a time. Do not read protected review before saving a genuine attempt. Every lesson follows the same beginner-first sequence even when the engineering idea becomes more technical.

Daily lesson loop

1) why it matters -> 2) what you learn -> 3) prerequisite -> 4) picture it -> 5) new words -> 6) useful visual/source -> 7) follow once -> 8) expected result -> 9) why it worked -> 10) mistakes -> 11) recovery -> 12) independent task -> 13) validate -> 14) teach it back -> 15) protected review + changed retry -> 16) evidence/mastery + exact next route.

Prerequisite check

Complete M02 SQL Foundations. You should already write SELECT/WHERE/GROUP BY/HAVING, simple CASE/subqueries, and validate row counts and grain.

Attempt protection

A protected review is repair material, not a shortcut. Save the first attempt, open only the matching review if needed, record the repair, close review, then solve a changed retry from a clean state.

Contents / Index

Every formal lesson and its actual page number in this exact rebuilt book.

DE03-01 - INNER JOIN and LEFT JOIN	5
DE03-02 - Cardinality, row multiplication and join validation	7
DE03-03 - UNION and UNION ALL	9
DE03-04 - String functions for cleaning and standardization	11
DE03-05 - Dates, timestamps and time analysis	13
DE03-06 - Subqueries and nested logic	15
DE03-07 - CTEs and readable multi-step SQL	17
DE03-08 - Window functions and PARTITION BY	19
DE03-09 - ROW_NUMBER, ranking and LAG/LEAD	21
DE03-10 - Advanced aggregation and multi-grain reasoning	23
DE03-11 - PostgreSQL setup, schemas and dialect awareness	25
DE03-12 - Constraints and transactions	27
DE03-13 - Indexes and EXPLAIN basics	30
DE03-14 - Views and reusable database logic	33
DE03-15 - Integrated unseen SQL challenge	35
DE03-16 - Timed SQL interview set and explanation	37

Prerequisite, tool and outcome map

ASSUMED

Complete M02 SQL Foundations. You should already write SELECT/WHERE/GROUP BY/HAVING, simple CASE/subqueries, and validate row counts and grain.

MODULE PURPOSE

Control joins, multi-step SQL, windows, PostgreSQL concepts and query reasoning.

EXIT EVIDENCE

Module practical + fresh retest + saved evidence; no critical unresolved failure.

Protected study route

Teaching -> assigned visual/source when useful -> follow once -> close worked material -> independent attempt -> validate -> save evidence -> teach it back. If weak: matching protected review -> targeted repair -> close review -> changed fresh retry -> revalidate.

DE03-01 - INNER JOIN and LEFT JOIN

1. Why it matters

Join related tables while choosing INNER vs LEFT from the business rule and stating the result grain before trusting the output. This matters because join, transaction and database decisions can silently multiply, drop or corrupt rows that every downstream result depends on.

2. What you are learning

Join related tables while choosing INNER vs LEFT from the business rule and stating the result grain before trusting the output.

3. Prerequisite / what you already know

Complete M02 SQL Foundations. You should already write SELECT/WHERE/GROUP BY/HAVING, simple CASE/subqueries, and validate row counts and grain.

4. Picture it in your head

Imagine two stacks of cards connected by matching ID badges. INNER JOIN keeps only matched pairs; LEFT JOIN keeps every left card and leaves right-side fields NULL when no partner exists.

5. New words before we use them

A JOIN combines rows through a relationship. INNER JOIN keeps matches only. LEFT JOIN preserves every left-side row and supplies NULLs when the right side has no match.

6. Useful video / source

Source: Alex The Analyst — SQL joins

Assigned scope: 51:10–1:08:04

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

OPEN VIDEO / SOURCE - CLICK HERE ([youtube.com](https://www.youtube.com/watch?v=OT1RERkflNQ))

URL: <https://www.youtube.com/watch?v=OT1RERkflNQ>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

```
SELECT o.order_id, o.customer_id, c.customer_name, c.tier FROM sales_orders o JOIN customers c ON  
c.customer_id=o.customer_id;
```

```
SELECT o.order_id, r.return_id FROM sales_orders o LEFT JOIN returns r ON r.order_id=o.order_id;
```

Run both joins. Before each one, say the left input grain, right input grain and intended output grain. Count unmatched return rows in the LEFT JOIN.

8. Expected result / what you should see

Customer join returns 30 order rows because customers is unique by customer_id. LEFT JOIN returns all 30 orders; exactly 5

- orders have a return_id in this lab.

9. Why it worked

The relationship is trustworthy because the join key and expected cardinality are stated before interpreting the output, and row/total controls can expose fan-out or dropped unmatched rows. Your independent check is: No non-West/non-Completed rows in the first query. The proof is row/key/grain reconciliation, not query complexity or successful execution.

10. Common beginner mistakes

1. Calling INNER JOIN and LEFT JOIN complete because the command or tool finished. 2. Skipping the independent control: No non-West/non-Completed rows in the first query. 3. Changing several things after a failure instead of isolating the first broken step. 4. Overwriting the failed attempt instead of saving it as repair evidence.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: Imagine two stacks of cards connected by matching ID badges. 3. Run the earliest independent control: No non-West/non-Completed rows in the first query. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

List Completed West orders with order_id, customer_name, tier and net_sales. Then LEFT JOIN returns and show all Completed orders with optional return_id.

13. Validate your result

No non-West/non-Completed rows in the first query. Second query preserves the Completed order count; returned

- orders have return_id and others are NULL.

14. Teach it back in your own words

Without notes, explain INNER JOIN and LEFT JOIN in your own words. Start with the purpose: Join related tables while choosing INNER vs LEFT from the business rule and stating the result grain before trusting the output. Then name one realistic failure mode and explain how you would prove the result using this check: No non-West/non-Completed rows in the first query.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for INNER JOIN and LEFT JOIN with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE03-02.

DE03-02 - Cardinality, row multiplication and join validation

1. Why it matters

Predict one-to-one/one-to-many join behavior and detect row multiplication before it corrupts counts or money. This matters because join, transaction and database decisions can silently multiply, drop or corrupt rows that every downstream result depends on.

2. What you are learning

Predict one-to-one/one-to-many join behavior and detect row multiplication before it corrupts counts or money.

3. Prerequisite / what you already know

Prerequisite: DE03-01 mastered with saved evidence.

4. Picture it in your head

Picture one customer card connected to many order cards. Cardinality predicts how many matches each side can produce; unexpected one-to-many links create fan-out and multiply rows.

5. New words before we use them

Cardinality describes how many rows on one side can match the other. A syntactically valid join can still be logically wrong. Validate row count, key uniqueness and at least one numeric total before and after risky joins.

6. Useful video / source

Source: Luke Barousse — Data Engineer Bootcamp (2026)

Assigned scope: 4:16:18 — Data Modeling & JOINS

How to use it: Use the join/modeling block for the visual mental model. Course task remains authoritative for cardinality, row multiplication and validation.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=ol9_NnC9-cc)

URL: https://www.youtube.com/watch?v=ol9_NnC9-cc

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

```
SELECT COUNT(*) FROM sales_orders;
```

```
-- 30 SELECT COUNT(*) FROM sales_orders o LEFT JOIN customer_contacts cc ON cc.customer_id=o.customer_id;
```

```
-- 35 Join sales_orders to customer_contacts by customer_id. Find order_ids that appear more than once and connect the extra rows to customers having multiple contact types.
```

8. Expected result / what you should see

Order rows = 30 before; 35 after the raw contact join. The five extra rows are expected fan-out from multi-contact customers,

- not five new orders.

9. Why it worked

The relationship is trustworthy because the join key and expected cardinality are stated before interpreting the output, and row/total controls can expose fan-out or dropped unmatched rows. Your independent check is: Output key order_id remains unique; row count is 30; contact_count can exceed 1 without duplicating orders. The proof is row/key/grain reconciliation, not query complexity or successful execution.

10. Common beginner mistakes

1. Calling Cardinality, row multiplication and join validation complete because the command or tool finished. 2. Skipping the independent control: Output key order_id remains unique; row count is 30; contact_count can exceed 1 without duplicating orders. 3. Changing several things after a failure instead of isolating the first broken step. 4. Overwriting the failed attempt instead of saving it as repair evidence.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: Picture one customer card connected to many order cards. 3. Run the earliest independent control: Output key order_id remains unique; row count is 30; contact_count can exceed 1 without duplicating orders. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Build a one-row-per-customer contact_count CTE/subquery, join it to orders, and prove the order row count stays 30.

13. Validate your result

Output key order_id remains unique; row count is 30; contact_count can exceed 1 without duplicating orders.

14. Teach it back in your own words

Without notes, explain Cardinality, row multiplication and join validation in your own words. Start with the purpose: Predict one-to-one/one-to-many join behavior and detect row multiplication before it corrupts counts or money. Then name one realistic failure mode and explain how you would prove the result using this check: Output key order_id remains unique; row count is 30; contact_count can exceed 1 without duplicating orders.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Cardinality, row multiplication and join validation with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE03-03.

DE03-03 - UNION and UNION ALL

1. Why it matters

Stack compatible result sets vertically and choose UNION vs UNION ALL intentionally. This matters because join, transaction and database decisions can silently multiply, drop or corrupt rows that every downstream result depends on.

2. What you are learning

Stack compatible result sets vertically and choose UNION vs UNION ALL intentionally.

3. Prerequisite / what you already know

Prerequisite: DE03-02 mastered with saved evidence.

4. Picture it in your head

UNION stacks result sets vertically like placing one same-shaped sheet under another. UNION ALL keeps every row; UNION also performs duplicate removal, which changes both meaning and cost.

5. New words before we use them

UNION ALL appends compatible result rows and keeps duplicates. UNION additionally removes duplicate result rows. UNION adds rows; JOIN adds columns.

6. Useful video / source

Source: Alex The Analyst — UNION

Assigned scope: 1:08:04–1:15:10

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=OT1RErkfLNQ)

URL: <https://www.youtube.com/watch?v=OT1RErkfLNQ>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

```
SELECT order_id AS event_id, customer_id, order_date AS event_date, '
Order'
```

```
AS event_type FROM sales_orders WHERE status='
Completed'
```

```
UNION ALL SELECT ticket_id, customer_id, DATE(created_at), '
Ticket'
```

```
FROM support_tickets;
```

Build the combined order/ticket activity stream. Add ORDER BY event_date,event_id after the UNION ALL.

8. Expected result / what you should see

The stream has 27 Completed-order events plus 10 ticket events = 37 rows because the event_id/type design keeps them

- distinct.

9. Why it worked

This works because stack compatible result sets vertically and choose UNION vs UNION ALL intentionally is made explicit before the result is accepted. Your independent check is: You can explain exactly which duplicate combination UNION removed and why. The proof is row/key/grain reconciliation, not query complexity or successful execution.

10. Common beginner mistakes

1. Calling UNION and UNION ALL complete because the command or tool finished. 2. Skipping the independent control: You can explain exactly which duplicate combination UNION removed and why. 3. Changing several things after a failure instead of isolating the first broken step. 4. Overwriting the failed attempt instead of saving it as repair evidence.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: UNION stacks result sets vertically like placing one same-shaped sheet under another. 3. Run the earliest independent control: You can explain exactly which duplicate combination UNION removed and why. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Create a second two-branch result with intentionally duplicated labels, compare UNION with UNION ALL, and record the row-count difference.

13. Validate your result

You can explain exactly which duplicate combination UNION removed and why.

14. Teach it back in your own words

Without notes, explain UNION and UNION ALL in your own words. Start with the purpose: Stack compatible result sets vertically and choose UNION vs UNION ALL intentionally. Then name one realistic failure mode and explain how you would prove the result using this check: You can explain exactly which duplicate combination UNION removed and why.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for UNION and UNION ALL with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE03-04.

DE03-04 - String functions for cleaning and standardization

1. Why it matters

Use TRIM/UPPER/LOWER/CONCAT and string matching without hiding the original value or overwriting source data. This matters because join, transaction and database decisions can silently multiply, drop or corrupt rows that every downstream result depends on.

2. What you are learning

Use TRIM/UPPER/LOWER/CONCAT and string matching without hiding the original value or overwriting source data.

3. Prerequisite / what you already know

Prerequisite: DE03-03 mastered with saved evidence.

4. Picture it in your head

A formula is a recipe attached to a cell: it takes referenced ingredients, performs an operation, and recalculates when the ingredients change.

5. New words before we use them

String functions can create cleaned/display values in a query. Keep the raw field beside transformed output while validating. A presentation cleanup is not proof that source data is correct.

6. Useful video / source

Source: Alex The Analyst — string functions

Assigned scope: 1:15:10–1:26:39

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=OT1RErkfLNQ)

URL: <https://www.youtube.com/watch?v=OT1RErkfLNQ>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

```
SELECT customer_id, customer_name, UPPER(customer_name) AS name_upper, CONCAT(customer_name, '
_ '
,tier) AS label FROM customers WHERE customer_name LIKE '
%Retail%'
;
```

Run the example and identify every Retail customer. Add a TRIM demo on a literal string with spaces before touching source fields.

8. Expected result / what you should see

Expected names include Ravi Retail, Om Retail, Ishita Retail, Saanvi Retail and Sara Retail.

9. Why it worked

The program is trustworthy because input/state changes are explicit and the output is checked against a known condition rather than accepting “the script ran” as correctness. Your independent check is: Raw and transformed names remain visible; the filter returns only names matching the intended text rule. The proof is row/key/grain reconciliation, not query complexity or successful execution.

10. Common beginner mistakes

1. Calling String functions for cleaning and standardization complete because the command or tool finished. 2. Skipping the independent control: Raw and transformed names remain visible; the filter returns only names matching the intended text rule. 3. Changing several things after a failure instead of isolating the first broken step. 4. Overwriting the failed attempt instead of saving it as repair evidence.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: A formula is a recipe attached to a cell: it takes referenced ingredients, performs an operation, and recalculates when the ingredients change. 3. Run the earliest independent control: Raw and transformed names remain visible; the filter returns only names matching the intended text rule. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Create customer_name_clean and a display label while preserving raw customer_name. Filter names containing Stores or Traders.

13. Validate your result

Raw and transformed names remain visible; the filter returns only names matching the intended text rule.

14. Teach it back in your own words

Without notes, explain String functions for cleaning and standardization in your own words. Start with the purpose: Use TRIM/UPPER/LOWER/CONCAT and string matching without hiding the original value or overwriting source data. Then name one realistic failure mode and explain how you would prove the result using this check: Raw and transformed names remain visible; the filter returns only names matching the intended text rule.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for String functions for cleaning and standardization with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE03-05.

DE03-05 - Dates, timestamps and time analysis

1. Why it matters

Calculate date differences, extract time parts and explain why date, datetime/timestamp and timezone meaning must be deliberate. This matters because join, transaction and database decisions can silently multiply, drop or corrupt rows that every downstream result depends on.

2. What you are learning

Calculate date differences, extract time parts and explain why date, datetime/timestamp and timezone meaning must be deliberate.

3. Prerequisite / what you already know

Prerequisite: DE03-04 mastered with saved evidence.

4. Picture it in your head

Picture every timestamp on a timeline with an explicit clock and time zone. Truncating, shifting, or comparing time changes the question, so boundaries must be stated.

5. New words before we use them

Dates represent calendar dates; timestamps/datetimes represent moments or local date-times depending on the system. DATEDIFF can answer elapsed-day questions in this MySQL lab. Real systems need an explicit timezone contract before comparing event times.

6. Useful video / source

Source: Luke Barousse — Data Engineer Bootcamp (2026)

Assigned scope: 10:16:15 — Functions: Date, Text & NULL

How to use it: Watch the date/time portion, reproduce examples, then solve the course timestamp/time analysis independently.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=ol9_NnC9-cc)

URL: https://www.youtube.com/watch?v=ol9_NnC9-cc

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

```
SELECT o.order_id, o.order_date, r.return_date, DATEDIFF(r.return_date,o.order_date) AS days_to_return FROM
sales_orders o JOIN returns r ON r.order_id=o.order_id ORDER BY days_to_return DESC;
```

Calculate days_to_return for all returned orders and inspect YEAR/MONTH/DAY on order_date.

8. Expected result / what you should see

Five returned orders are present; every days_to_return is non-negative in the supplied lab.

9. Why it worked

This works because calculate date differences, extract time parts and explain why date, datetime/timestamp and timezone meaning must be deliberate is made explicit before the result is accepted. Your independent check is: Daily groups reconcile back to the Completed net-sales total; timezone limitation is stated rather than guessed. The proof is row/key/grain reconciliation, not query complexity or successful execution.

10. Common beginner mistakes

1. Calling Dates, timestamps and time analysis complete because the command or tool finished. 2. Skipping the independent control: Daily groups reconcile back to the Completed net-sales total; timezone limitation is stated rather than guessed. 3. Changing several things after a failure instead of isolating the first broken step. 4. Overwriting the failed attempt instead of saving it as repair evidence.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: Picture every timestamp on a timeline with an explicit clock and time zone. 3. Run the earliest independent control: Daily groups reconcile back to the Completed net-sales total; timezone limitation is stated rather than guessed. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Summarize Completed net_sales by order_date and identify the dates with the highest daily total. Explain what would change if order timestamps arrived from multiple timezones.

13. Validate your result

Daily groups reconcile back to the Completed net-sales total; timezone limitation is stated rather than guessed.

14. Teach it back in your own words

Without notes, explain Dates, timestamps and time analysis in your own words. Start with the purpose: Calculate date differences, extract time parts and explain why date, datetime/timestamp and timezone meaning must be deliberate. Then name one realistic failure mode and explain how you would prove the result using this check: Daily groups reconcile back to the Completed net-sales total; timezone limitation is stated rather than guessed.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Dates, timestamps and time analysis with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE03-06.

DE03-06 - Subqueries and nested logic

1. Why it matters

Use scalar/list subqueries while keeping each nested level independently testable. This matters because join, transaction and database decisions can silently multiply, drop or corrupt rows that every downstream result depends on.

2. What you are learning

Use scalar/list subqueries while keeping each nested level independently testable.

3. Prerequisite / what you already know

Prerequisite: DE03-05 mastered with saved evidence.

4. Picture it in your head

Think of nested SQL as boxes inside boxes. Validate the inner box first, then use its known output in the outer query so a wrong inner result cannot hide inside a larger statement.

5. New words before we use them

A subquery is a query used inside another query. Debug the inner level first. Deep nesting is not a badge of sophistication; readable grain and validation matter more.

6. Useful video / source

Source: Alex The Analyst — subqueries

Assigned scope: 1:35:16–1:45:58

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

OPEN VIDEO / SOURCE - CLICK HERE ([youtube.com](https://www.youtube.com/watch?v=OT1RErkfLNQ))

URL: <https://www.youtube.com/watch?v=OT1RErkfLNQ>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

```
SELECT order_id, qty*unit_price*(1-discount_pct) AS net_sales FROM sales_orders WHERE status='Completed'
```

```
AND qty*unit_price*(1-discount_pct) >
```

```
( SELECT AVG(qty*unit_price*(1-discount_pct)) FROM sales_orders WHERE status='Completed'
```

```
);
```

Run the inner average independently, then the full above-average Completed-order query.

8. Expected result / what you should see

The inner subquery returns one scalar. Outer rows are all Completed and each net_sales is above that scalar.

9. Why it worked

This works because use scalar/list subqueries while keeping each nested level independently testable is made explicit before the result is accepted. Your independent check is: State inner and outer grain separately and reconcile customer totals to overall Completed net_sales. The proof is row/key/grain reconciliation, not query complexity or successful execution.

10. Common beginner mistakes

1. Calling Subqueries and nested logic complete because the command or tool finished. 2. Skipping the independent control: State inner and outer grain separately and reconcile customer totals to overall Completed net_sales. 3. Changing several things after a failure instead of isolating the first broken step. 4. Overwriting the failed attempt instead of saving it as repair evidence.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: Think of nested SQL as boxes inside boxes. 3. Run the earliest independent control: State inner and outer grain separately and reconcile customer totals to overall Completed net_sales. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Find customers whose Completed net_sales total is above the average customer Completed total. Build/validate the customer-total layer first.

13. Validate your result

State inner and outer grain separately and reconcile customer totals to overall Completed net_sales.

14. Teach it back in your own words

Without notes, explain Subqueries and nested logic in your own words. Start with the purpose: Use scalar/list subqueries while keeping each nested level independently testable. Then name one realistic failure mode and explain how you would prove the result using this check: State inner and outer grain separately and reconcile customer totals to overall Completed net_sales.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Subqueries and nested logic with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE03-07.

DE03-07 - CTEs and readable multi-step SQL

1. Why it matters

Break multi-step SQL into named CTEs and state the grain at each stage. This matters because join, transaction and database decisions can silently multiply, drop or corrupt rows that every downstream result depends on.

2. What you are learning

Break multi-step SQL into named CTEs and state the grain at each stage.

3. Prerequisite / what you already know

Prerequisite: DE03-06 mastered with saved evidence.

4. Picture it in your head

A CTE is a named stage on a whiteboard: source -> filtered -> enriched -> summarized. Each stage can be read and checked before the final SELECT.

5. New words before we use them

A CTE gives a query step a name. Its primary value here is readability and inspectable grain, not automatic performance. Each CTE should have a clear purpose and row meaning.

6. Useful video / source

Source: Alex The Analyst — CTEs

Assigned scope: 1:58:55–2:09:07

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=OT1RErkfLNQ)

URL: <https://www.youtube.com/watch?v=OT1RErkfLNQ>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

```
WITH order_sales AS ( SELECT order_id, customer_id, region, status, qty*unit_price*(1-discount_pct) AS net_sales FROM sales_orders ), customer_sales AS ( SELECT customer_id, SUM(net_sales) AS customer_net FROM order_sales WHERE status='Completed'
```

```
GROUP BY customer_id ) SELECT * FROM customer_sales ORDER BY customer_net DESC;
```

Build order_sales then customer_sales. Run each stage separately during development and record row counts.

8. Expected result / what you should see

order_sales = one row/order, 30 rows. customer_sales = one row/customer with Completed activity. Its summed

- customer_net reconciles to Completed net sales.

9. Why it worked

The query is trustworthy because the business question, row grain and transformation stages are explicit, then an independent row/key/total check challenges a plausible-but-wrong result. Your independent check is: Every CTE has a written grain comment and totals reconcile. The proof is row/key/grain reconciliation, not query complexity or successful execution.

10. Common beginner mistakes

1. Calling CTEs and readable multi-step SQL complete because the command or tool finished. 2. Skipping the independent control: Every CTE has a written grain comment and totals reconcile. 3. Changing several things after a failure instead of isolating the first broken step. 4. Overwriting the failed attempt instead of saving it as repair evidence.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: A CTE is a named stage on a whiteboard: source -> filtered -> enriched -> summarized. 3. Run the earliest independent control: Every CTE has a written grain comment and totals reconcile. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Rewrite your DE03-06 customer-above-average solution with named CTEs and add a final validation CTE/result.

13. Validate your result

Every CTE has a written grain comment and totals reconcile.

14. Teach it back in your own words

Without notes, explain CTEs and readable multi-step SQL in your own words. Start with the purpose: Break multi-step SQL into named CTEs and state the grain at each stage. Then name one realistic failure mode and explain how you would prove the result using this check: Every CTE has a written grain comment and totals reconcile.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for CTEs and readable multi-step SQL with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE03-08.

DE03-08 - Window functions and PARTITION BY

1. Why it matters

Add group context to detail rows with window functions while distinguishing window grain from GROUP BY grain. This matters because join, transaction and database decisions can silently multiply, drop or corrupt rows that every downstream result depends on.

2. What you are learning

Add group context to detail rows with window functions while distinguishing window grain from GROUP BY grain.

3. Prerequisite / what you already know

Prerequisite: DE03-07 mastered with saved evidence.

4. Picture it in your head

A formula is a recipe attached to a cell: it takes referenced ingredients, performs an operation, and recalculates when the ingredients change.

5. New words before we use them

GROUP BY collapses rows to the group grain. A window calculation can preserve detail rows and attach group-level context. PARTITION BY defines independent calculation groups.

6. Useful video / source

Source: Alex The Analyst — window functions

Assigned scope: 1:45:58–1:58:55

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=OT1RErkfLNQ)

URL: <https://www.youtube.com/watch?v=OT1RErkfLNQ>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

WITH x AS (SELECT order_id,region,qty*unit_price*(1-discount_pct) AS net_sales FROM sales_orders WHERE status='Completed'

) SELECT order_id,region,net_sales, AVG(net_sales) OVER(PARTITION BY region) AS region_avg_sales FROM x;

Add region average net_sales beside every Completed order. Compare row count to a GROUP BY region query.

8. Expected result / what you should see

Window result preserves 27 Completed detail rows; GROUP BY region produces 4 rows.

9. Why it worked

The calculation works because the window defines peer groups/order while preserving row grain; a separate grouped or row-count check confirms that detail rows were not accidentally collapsed or duplicated. Your independent check is: Detail row count stays 27; a separate GROUP BY customer query matches the window totals. The proof is row/key/grain reconciliation, not query complexity or successful execution.

10. Common beginner mistakes

1. Calling Window functions and PARTITION BY complete because the command or tool finished. 2. Skipping the independent control: Detail row count stays 27; a separate GROUP BY customer query matches the window totals. 3. Changing several things after a failure instead of isolating the first broken step. 4. Overwriting the failed attempt instead of saving it as repair evidence.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: A formula is a recipe attached to a cell: it takes referenced ingredients, performs an operation, and recalculates when the ingredients change. 3. Run the earliest independent control: Detail row count stays 27; a separate GROUP BY customer query matches the window totals. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Add each customer's Completed order count and customer net-sales total beside each of their Completed orders.

13. Validate your result

Detail row count stays 27; a separate GROUP BY customer query matches the window totals.

14. Teach it back in your own words

Without notes, explain Window functions and PARTITION BY in your own words. Start with the purpose: Add group context to detail rows with window functions while distinguishing window grain from GROUP BY grain. Then name one realistic failure mode and explain how you would prove the result using this check: Detail row count stays 27; a separate GROUP BY customer query matches the window totals.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Window functions and PARTITION BY with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE03-09.

DE03-09 - ROW_NUMBER, ranking and LAG/LEAD

1. Why it matters

Use deterministic ROW_NUMBER/ranking and LAG/LEAD for latest-row, top-N and previous/next-row analysis. This matters because join, transaction and database decisions can silently multiply, drop or corrupt rows that every downstream result depends on.

2. What you are learning

Use deterministic ROW_NUMBER/ranking and LAG/LEAD for latest-row, top-N and previous/next-row analysis.

3. Prerequisite / what you already know

Prerequisite: DE03-08 mastered with saved evidence.

4. Picture it in your head

Picture a leaderboard with each row still present. ROW_NUMBER gives positions, while LAG/LEAD lets a row look at its previous/next neighbor without collapsing the detail.

5. New words before we use them

ROW_NUMBER assigns a unique sequence based on the ORDER BY you define. RANK/DENSE_RANK treat ties differently. LAG/LEAD reaches a previous/next row within a partition/order.

6. Useful video / source

Source: Alex The Analyst — ranking/window functions

Assigned scope: 1:45:58–1:58:55

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=OT1RErkfLNQ)

URL: <https://www.youtube.com/watch?v=OT1RErkfLNQ>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

WITH t AS (SELECT ticket_id, customer_id, created_at, status, ROW_NUMBER() OVER(PARTITION BY customer_id ORDER BY created_at DESC, ticket_id DESC) AS rn FROM support_tickets) SELECT * FROM t WHERE rn=1;
Return latest support ticket per customer. Then add LAG(order_date) to each customer's orders.

8. Expected result / what you should see

Latest-ticket query returns one row per customer who has tickets. Sequence has an explicit ticket_id tie-breaker.

9. Why it worked

The calculation works because the window defines peer groups/order while preserving row grain; a separate grouped or row-count check confirms that detail rows were not accidentally collapsed or duplicated. Your independent check is: At most 2 rows/region; order ranking is reproducible; compare each region maximum manually/separately. The proof is row/key/grain reconciliation, not query complexity or successful execution.

10. Common beginner mistakes

1. Calling ROW_NUMBER, ranking and LAG/LEAD complete because the command or tool finished. 2. Skipping the independent control: At most 2 rows/region; order ranking is reproducible; compare each region maximum manually/separately. 3. Changing several things after a failure instead of isolating the first broken step. 4. Overwriting the failed attempt instead of saving it as repair evidence.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: Picture a leaderboard with each row still present. 3. Run the earliest independent control: At most 2 rows/region; order ranking is reproducible; compare each region maximum manually/separately. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Rank Completed orders by net_sales within region and return top 2 per region with deterministic order_id tie-breaker.

13. Validate your result

At most 2 rows/region; order ranking is reproducible; compare each region maximum manually/separately.

14. Teach it back in your own words

Without notes, explain ROW_NUMBER, ranking and LAG/LEAD in your own words. Start with the purpose: Use deterministic ROW_NUMBER/ranking and LAG/LEAD for latest-row, top-N and previous/next-row analysis. Then name one realistic failure mode and explain how you would prove the result using this check: At most 2 rows/region; order ranking is reproducible; compare each region maximum manually/separately.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for ROW_NUMBER, ranking and LAG/LEAD with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE03-10.

DE03-10 - Advanced aggregation and multi-grain reasoning

1. Why it matters

Combine metrics from different source grains without multiplying them and defend the chosen final grain. This matters because join, transaction and database decisions can silently multiply, drop or corrupt rows that every downstream result depends on.

2. What you are learning

Combine metrics from different source grains without multiplying them and defend the chosen final grain.

3. Prerequisite / what you already know

Prerequisite: DE03-09 mastered with saved evidence.

4. Picture it in your head

Think of grain as camera zoom: “one row per order” and “one row per order item” are different zoom levels. A calculation is trustworthy only when you know which level the row represents.

5. New words before we use them

Orders, returns and tickets are not automatically the same grain. Compute each metric at a stable grain first, then join summaries at the intended output grain. COUNT(DISTINCT .) can sometimes protect a count, but it does not magically repair multiplied monetary sums.

6. Useful video / source

Source: Luke Barousse — SQL for Data Analytics

Assigned scope: 37:44–1:04:47 — aggregation through statistical aggregation; 1:04:47 onward segmentation

How to use it: Use these blocks to strengthen aggregation fluency. The Teaching Book adds multi-grain engineering reasoning and validation.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=7mz73uXD9DA)

URL: <https://www.youtube.com/watch?v=7mz73uXD9DA>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

```
WITH order_region AS ( SELECT region,COUNT(*) AS orders_count,SUM(qty*unit_price*(1-discount_pct)) AS net_sales
FROM sales_orders WHERE status='
Completed'
```

```
GROUP BY region ), return_region AS ( SELECT o.region,COUNT(*) AS returned_orders FROM returns r JOIN
sales_orders o ON o.order_id=r.order_id GROUP BY o.region ) SELECT o.*,COALESCE(r.returned_orders,0) AS
returned_orders FROM order_region o LEFT JOIN return_region r USING(region);
```

Build order and return metrics separately by region, then combine them. Explain why directly joining orders to tickets by customer_id could multiply both sides.

8. Expected result / what you should see

Final result is exactly one row/region. Sum of regional net_sales reconciles to Completed net_sales. Returned-order counts

- sum to 5.

9. Why it worked

The calculation is trustworthy because the output grain is declared before aggregation and the grouped result is reconciled to an independent detail-level control. Your independent check is: 4 final rows; each component total reconciles independently; document any customer-to-region assumption. The proof is row/key/grain reconciliation, not query complexity or successful execution.

10. Common beginner mistakes

1. Calling Advanced aggregation and multi-grain reasoning complete because the command or tool finished. 2. Skipping the independent control: 4 final rows; each component total reconciles independently; document any customer-to-region assumption. 3. Changing several things after a failure instead of isolating the first broken step. 4. Overwriting the failed attempt instead of saving it as repair evidence.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: Think of grain as camera zoom: “one row per order” and “one row per order item” are different zoom levels. 3. Run the earliest independent control: 4 final rows; each component total reconciles independently; document any customer-to-region assumption. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Create region metrics: Completed orders, Completed net_sales, returned orders, ticket count. Use separate grain-safe CTEs and combine at one row/region.

13. Validate your result

4 final rows; each component total reconciles independently; document any customer-to-region assumption.

14. Teach it back in your own words

Without notes, explain Advanced aggregation and multi-grain reasoning in your own words. Start with the purpose: Combine metrics from different source grains without multiplying them and defend the chosen final grain. Then name one realistic failure mode and explain how you would prove the result using this check: 4 final rows; each component total reconciles independently; document any customer-to-region assumption.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Advanced aggregation and multi-grain reasoning with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE03-11.

DE03-11 - PostgreSQL setup, schemas and dialect awareness

1. Why it matters

Install/verify PostgreSQL when this lesson begins, connect with psql or pgAdmin, create/use a schema, and recognize that SQL dialect/tooling differs from MySQL. This matters because join, transaction and database decisions can silently multiply, drop or corrupt rows that every downstream result depends on.

2. What you are learning

Install/verify PostgreSQL when this lesson begins, connect with psql or pgAdmin, create/use a schema, and recognize that SQL dialect/tooling differs from MySQL.

3. Prerequisite / what you already know

Prerequisite: DE03-10 mastered with saved evidence.

4. Picture it in your head

Picture one PostgreSQL server as a building: databases/schemas organize rooms, tables live inside them, and PostgreSQL syntax is the local language. Always know which room and dialect your query is using.

5. New words before we use them

PostgreSQL is a separate database system from MySQL. Common SQL concepts transfer, but setup, functions, types and commands can differ. A schema is a namespace inside a PostgreSQL database.

6. Useful video / source

Source: freeCodeCamp — PostgreSQL setup

Assigned scope: 0:00–17:12

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=SpflwIAYaKk)

URL: <https://www.youtube.com/watch?v=SpflwIAYaKk>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

```
SELECT version();
```

```
CREATE SCHEMA IF NOT EXISTS m03pg;
```

```
SET search_path TO m03pg, public;
```

```
SELECT current_schema();
```

Install/verify the current supported PostgreSQL release from the official project when you reach this lesson. Create database m03_de (or use an existing course DB), run M03_POSTGRES_SQL_MINI_LAB_SETUP.sql, then run SELECT version() and SHOW search_path.

8. Expected result / what you should see

You can connect successfully, identify the server version you actually observe, and query m03pg.orders. The mini-lab reports

- 8 customers and 12 orders.

9. Why it worked

The result is trustworthy because structure is treated as a contract rather than inferred casually; the validation checks fields/types/constraints against the declared shape. Your independent check is: Query runs in PostgreSQL, not MySQL; result has 12 rows; dialect notes are evidence-based rather than invented. The proof is row/key/grain reconciliation, not query complexity or successful execution.

10. Common beginner mistakes

1. Calling PostgreSQL setup, schemas and dialect awareness complete because the command or tool finished. 2. Skipping the independent control: Query runs in PostgreSQL, not MySQL; result has 12 rows; dialect notes are evidence-based rather than invented. 3. Changing several things after a failure instead of isolating the first broken step. 4. Overwriting the failed attempt instead of saving it as repair evidence.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: Picture one PostgreSQL server as a building: databases/schemas organize rooms, tables live inside them, and PostgreSQL syntax is the local language. 3. Run the earliest independent control: Query runs in PostgreSQL, not MySQL; result has 12 rows; dialect notes are evidence-based rather than invented. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

In a fresh query tab, set search_path and return order_id, customer_id, status from m03pg.orders. Write two syntax/function differences you have actually observed or verified between the MySQL and PostgreSQL labs.

13. Validate your result

Query runs in PostgreSQL, not MySQL; result has 12 rows; dialect notes are evidence-based rather than invented.

14. Teach it back in your own words

Without notes, explain PostgreSQL setup, schemas and dialect awareness in your own words. Start with the purpose: Install/verify PostgreSQL when this lesson begins, connect with psql or pgAdmin, create/use a schema, and recognize that SQL dialect/tooling differs from MySQL. Then name one realistic failure mode and explain how you would prove the result using this check: Query runs in PostgreSQL, not MySQL; result has 12 rows; dialect notes are evidence-based rather than invented.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for PostgreSQL setup, schemas and dialect awareness with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE03-12.

DE03-12 - Constraints and transactions

1. Why it matters

Use primary/foreign/check/not-null constraints and prove COMMIT/ROLLBACK behavior without weakening integrity rules. This matters because join, transaction and database decisions can silently multiply, drop or corrupt rows that every downstream result depends on.

2. What you are learning

Use primary/foreign/check/not-null constraints and prove COMMIT/ROLLBACK behavior without weakening integrity rules.

3. Prerequisite / what you already know

Prerequisite: DE03-11 mastered with saved evidence.

4. Picture it in your head

Constraints are guardrails at the table boundary; a transaction is an all-or-nothing envelope around related changes. Either the complete valid set commits or it rolls back together.

5. New words before we use them

Constraints make invalid states harder to store. A transaction groups changes so they can commit together or roll back. When a constraint rejects a row, the correct response is to understand the rule/data - not remove the constraint to silence the error.

6. Useful video / source

Source: freeCodeCamp — PostgreSQL constraints/transactions

Assigned scope: 2:29:24–3:25:04

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=qw--VYLpxG4)

URL: <https://www.youtube.com/watch?v=qw--VYLpxG4>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

```
BEGIN;
UPDATE m03pg.orders SET status='
Cancelled'
WHERE order_id='
P1001'
;
SELECT order_id,status FROM m03pg.orders WHERE order_id='
P1001'
;
ROLLBACK;
SELECT order_id,status FROM m03pg.orders WHERE order_id='
P1001'
;
```

Run the rollback demonstration. Then deliberately attempt a duplicate order_id and an invalid status inside a safe transaction and capture the constraint errors.

8. Expected result / what you should see

P1001 returns to its original status after ROLLBACK. Invalid/duplicate rows do not remain stored.

9. Why it worked

The operation is trustworthy because related changes cross the commit boundary together; a before/after state check proves partial updates are not being mistaken for success. Your independent check is: Baseline row count restored after rollback; no invalid row persists. The proof is row/key/grain reconciliation, not query complexity or successful execution.

10. Common beginner mistakes

1. Calling Constraints and transactions complete because the command or tool finished. 2. Skipping the independent control: Baseline row count restored after rollback; no invalid row persists. 3. Changing several things after a failure instead of isolating the first broken step. 4. Overwriting the failed attempt instead of saving it as repair evidence.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: Constraints are guardrails at the table boundary; a transaction is an all-or-nothing envelope around related changes. 3. Run the earliest independent control: Baseline row count restored after rollback; no invalid row persists. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Insert one valid support_case inside BEGIN; inspect it; ROLLBACK; prove row count returns to the baseline. Explain which constraint protects each critical field.

13. Validate your result

Baseline row count restored after rollback; no invalid row persists.

14. Teach it back in your own words

Without notes, explain Constraints and transactions in your own words. Start with the purpose: Use primary/foreign/check/not-null constraints and prove COMMIT/ROLLBACK behavior without weakening integrity rules. Then name one realistic failure mode and explain how you would prove the result using this check: Baseline row count restored after rollback; no invalid row persists.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Constraints and transactions with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE03-13.

DE03-13 - Indexes and EXPLAIN basics

1. Why it matters

Create a query-driven index, read basic EXPLAIN evidence, and avoid claiming speedup that the small learning table cannot prove. This matters because join, transaction and database decisions can silently multiply, drop or corrupt rows that every downstream result depends on.

2. What you are learning

Create a query-driven index, read basic EXPLAIN evidence, and avoid claiming speedup that the small learning table cannot prove.

3. Prerequisite / what you already know

Prerequisite: DE03-12 mastered with saved evidence.

4. Picture it in your head

An index is like the index at the back of a book: it can help the database locate selective values without scanning every page. EXPLAIN is the route plan that shows how PostgreSQL intends to execute the query.

5. New words before we use them

An index is an additional structure that can help some lookups but costs storage/write maintenance. PostgreSQL chooses plans based on table size/statistics/cost. On tiny course tables, the planner may still prefer a sequential scan.

6. Useful video / source

Source: Luke Barousse — SQL for Data Analytics

Assigned scope: 5:10:55–5:42:10 — EXPLAIN and query optimization

How to use it: Watch the EXPLAIN/optimization block, then use the course PostgreSQL task to compare plans and explain what changed.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=7mz73uXD9DA)

URL: <https://www.youtube.com/watch?v=7mz73uXD9DA>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

```
EXPLAIN SELECT * FROM m03pg.orders WHERE customer_id='PC003'
```

```
;
```

```
CREATE INDEX IF NOT EXISTS idx_m03_orders_customer_id ON m03pg.orders(customer_id);
```

```
ANALYZE m03pg.orders;
```

```
EXPLAIN SELECT * FROM m03pg.orders WHERE customer_id='PC003'
```

```
;
```

Capture EXPLAIN before and after the index. Compare plan nodes/cost/rows estimates you actually see; do not require an Index Scan on a 12-row table.

8. Expected result / what you should see

Index exists in pg_indexes after creation. Before/after plans are saved. Any claim about planner choice matches observed

- output.

9. Why it worked

The reasoning separates correctness from performance: first prove the result, then inspect the planner/execution evidence to see whether the engine can avoid unnecessary work. Your independent check is: Evidence includes the actual index definition and plans; explanation does not equate “index exists” with “query is faster.”. The proof is row/key/grain reconciliation, not query complexity or successful execution.

10. Common beginner mistakes

1. Calling Indexes and EXPLAIN basics complete because the command or tool finished. 2. Skipping the independent control: Evidence includes the actual index definition and plans; explanation does not equate “index exists” with “query is faster.”. 3. Changing several things after a failure instead of isolating the first broken step. 4. Overwriting the failed attempt instead of saving it as repair evidence.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: An index is like the index at the back of a book: it can help the database locate selective values without scanning every page. 3. Run the earliest independent control: Evidence includes the actual index definition and plans; explanation does not equate “index exists” with “query is faster.” 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Choose one repeated filter/join column from the mini-lab, justify whether an index is sensible, create it only if justified, and save before/after EXPLAIN plus one downside.

13. Validate your result

Evidence includes the actual index definition and plans; explanation does not equate “index exists” with “query is faster.”

14. Teach it back in your own words

Without notes, explain Indexes and EXPLAIN basics in your own words. Start with the purpose: Create a query-driven index, read basic EXPLAIN evidence, and avoid claiming speedup that the small learning table cannot prove. Then name one realistic failure mode and explain how you would prove the result using this check: Evidence includes the actual index definition and plans; explanation does not equate “index exists” with “query is faster.”.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Indexes and EXPLAIN basics with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE03-14.

DE03-14 - Views and reusable database logic

1. Why it matters

Create a view for stable reusable query logic while explaining that a normal view stores a query definition, not an independent copy of the data. This matters because join, transaction and database decisions can silently multiply, drop or corrupt rows that every downstream result depends on.

2. What you are learning

Create a view for stable reusable query logic while explaining that a normal view stores a query definition, not an independent copy of the data.

3. Prerequisite / what you already know

Prerequisite: DE03-13 mastered with saved evidence.

4. Picture it in your head

A view is a saved query lens: consumers can query the named view without rewriting the logic, but the underlying tables and query definition still determine correctness.

5. New words before we use them

A normal PostgreSQL view gives a reusable named SELECT. It can simplify consumer queries and centralize logic, but it depends on underlying objects and permissions. A view is not automatically a performance optimization or a materialized dataset.

6. Useful video / source

Source: freeCodeCamp — PostgreSQL detailed course

Assigned scope: Use matching Views section; Teaching Book is authoritative

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

OPEN VIDEO / SOURCE - CLICK HERE (youtube.com)

URL: <https://www.youtube.com/watch?v=qw--VYLpxG4>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

```
CREATE OR REPLACE VIEW m03pg.v_completed_order_value AS SELECT
o.order_id,o.order_date,o.customer_id,c.customer_name, SUM(i.qty*i.unit_price) AS gross_sales FROM m03pg.orders o
JOIN m03pg.customers c USING(customer_id) JOIN m03pg.order_items i USING(order_id) WHERE o.status='
Completed'
GROUP BY o.order_id,o.order_date,o.customer_id,c.customer_name;
SELECT * FROM m03pg.v_completed_order_value;
```

Create the completed-order view, query it, then change one underlying row inside a transaction and observe the view before rolling back.

8. Expected result / what you should see

The view exposes only Completed orders and reflects underlying data because it is a normal view. Rollback returns results to

- baseline.

9. Why it worked

The query is trustworthy because the business question, row grain and transformation stages are explicit, then an independent row/key/total check challenges a plausible-but-wrong result. Your independent check is: One row/customer with activity; sum of gross_sales matches an independent aggregate over eligible source rows. The proof is row/key/grain reconciliation, not query complexity or successful execution.

10. Common beginner mistakes

1. Calling Views and reusable database logic complete because the command or tool finished. 2. Skipping the independent control: One row/customer with activity; sum of gross_sales matches an independent aggregate over eligible source rows. 3. Changing several things after a failure instead of isolating the first broken step. 4. Overwriting the failed attempt instead of saving it as repair evidence.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: A view is a saved query lens: consumers can query the named view without rewriting the logic, but the underlying tables and query definition still determine correctness. 3. Run the earliest independent control: One row/customer with activity; sum of gross_sales matches an independent aggregate over eligible source rows. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Create a view m03pg.v_customer_order_summary at one row/customer with order_count and gross_sales. State dependencies and one limitation.

13. Validate your result

One row/customer with activity; sum of gross_sales matches an independent aggregate over eligible source rows.

14. Teach it back in your own words

Without notes, explain Views and reusable database logic in your own words. Start with the purpose: Create a view for stable reusable query logic while explaining that a normal view stores a query definition, not an independent copy of the data. Then name one realistic failure mode and explain how you would prove the result using this check: One row/customer with activity; sum of gross_sales matches an independent aggregate over eligible source rows.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Views and reusable database logic with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE03-15.

DE03-15 - Integrated unseen SQL challenge

1. Why it matters

Combine grain/cardinality, CTE/window logic, validation and PostgreSQL database-thinking skills without tutorial support. This matters because join, transaction and database decisions can silently multiply, drop or corrupt rows that every downstream result depends on.

2. What you are learning

Combine grain/cardinality, CTE/window logic, validation and PostgreSQL database-thinking skills without tutorial support.

3. Prerequisite / what you already know

Prerequisite: DE03-14 mastered with saved evidence.

4. Picture it in your head

Treat the unseen challenge like a closed-book driving test: first restate the business question and grain, then build the query in checkable stages, then defend the result with independent controls.

5. New words before we use them

This is an assessment lesson, not another tutorial. A genuine attempt is required before review. You may use the database clients and the supplied source data, but keep videos, solution/review material and AI solving closed.

6. Useful video / source

Internal teaching route

Internal teaching route. This lesson is fully taught in this book and its bundled practice. Use the mental model and worked demonstration here; no filler external video is required for mastery.

7. Follow with me once

No worked solution is shown before attempt. Read the challenge brief, state grains/eligibility, build in small validated steps and preserve failed attempts. No guided solution. Planning scaffold only: 1) state desired output grain; 2) list source grains; 3) identify fan-out risks; 4) build/validate stages; 5) save result + controls; 6) explain limitations.

8. Expected result / what you should see

Challenge rubric requires correct grain, no unexplained multiplication, at least one CTE/window pattern, two independent • validations, and one PostgreSQL object/transaction/plan evidence item.

9. Why it worked

The query is trustworthy because the business question, row grain and transformation stages are explicit, then an independent row/key/total check challenges a plausible-but-wrong result. Your independent check is: All monetary totals reconcile, customer_id is unique in final customer table, latest-ticket logic is deterministic, and. The proof is row/key/grain reconciliation, not query complexity or successful execution.

10. Common beginner mistakes

1. Calling Integrated unseen SQL challenge complete because the command or tool finished. 2. Skipping the independent control: All monetary totals reconcile, customer_id is unique in final customer table, latest-ticket logic is deterministic, and. 3. Changing several things after a failure instead of isolating the first broken step. 4. Overwriting the failed attempt instead of saving it as repair evidence.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: Treat the unseen challenge like a closed-book driving test: first restate the business question and grain, then build the query in checkable stages, then defend the result with independent controls. 3. Run the earliest independent control: All monetary totals reconcile, customer_id is unique in final customer table, latest-ticket logic is deterministic, and 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Complete the M03 integrated challenge in 90 minutes. MySQL portion: build a one-row/customer priority table from orders + tickets without fan-out, with latest ticket and customer sales rank. PostgreSQL portion: prove one rollback, one justified index/EXPLAIN comparison and one reusable view.

13. Validate your result

All monetary totals reconcile, customer_id is unique in final customer table, latest-ticket logic is deterministic, and

- PostgreSQL evidence is observed/saved.

14. Teach it back in your own words

Without notes, explain Integrated unseen SQL challenge in your own words. Start with the purpose: Combine grain/cardinality, CTE/window logic, validation and PostgreSQL database-thinking skills without tutorial support. Then name one realistic failure mode and explain how you would prove the result using this check: All monetary totals reconcile, customer_id is unique in final customer table, latest-ticket logic is deterministic, and.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Integrated unseen SQL challenge with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE03-16.

DE03-16 - Timed SQL interview set and explanation

1. Why it matters

Solve a fresh timed SQL set and explain choices clearly under interview-like constraints. This matters because join, transaction and database decisions can silently multiply, drop or corrupt rows that every downstream result depends on.

2. What you are learning

Solve a fresh timed SQL set and explain choices clearly under interview-like constraints.

3. Prerequisite / what you already know

Prerequisite: DE03-15 mastered with saved evidence.

4. Picture it in your head

Picture the interview as two parallel tracks: solve the query and narrate your reasoning. A correct answer you cannot explain is weaker evidence than a structured approach with explicit checks.

5. New words before we use them

Timed work tests retrieval and reasoning, not memorization. Correctness includes grain and validation. A query that runs but answers the wrong question is not a pass.

6. Useful video / source

Internal teaching route

Internal teaching route. This lesson is fully taught in this book and its bundled practice. Use the mental model and worked demonstration here; no filler external video is required for mastery.

7. Follow with me once

No answer is exposed before attempt. Use a 45-minute timer for the SQL set plus 10-15 minutes for explanation/validation notes. No guided solution. Time allocation suggestion: 5 min read/grain, 30 min queries, 10 min validation, then explanation.

8. Expected result / what you should see

Pass target: $\geq 85/100$, no critical grain/cardinality/unsafe-transaction failure, and clear explanation of at least one

- window/CTE and one PostgreSQL database object.

9. Why it worked

The query is trustworthy because the business question, row grain and transformation stages are explicit, then an independent row/key/total check challenges a plausible-but-wrong result. Your independent check is: Save SQL, result controls, time used, confidence, errors and explanation. The proof is row/key/grain reconciliation, not query complexity or successful execution.

10. Common beginner mistakes

1. Calling Timed SQL interview set and explanation complete because the command or tool finished. 2. Skipping the independent control: Save SQL, result controls, time used, confidence, errors and explanation. 3. Changing several things after a failure instead of isolating the first broken step. 4. Overwriting the failed attempt instead of saving it as repair evidence.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: Picture the interview as two parallel tracks: solve the query and narrate your reasoning. 3. Run the earliest independent control: Save SQL, result controls, time used, confidence, errors and explanation. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Timed set: (1) diagnose a fan-out join; (2) top-2 Completed non-returned orders/region; (3) latest ticket/customer; (4) region net-sales + return rate safely; (5) explain transaction rollback and EXPLAIN evidence; (6) defend view vs table.

13. Validate your result

Save SQL, result controls, time used, confidence, errors and explanation. Review remains closed until the attempt is

- preserved.

14. Teach it back in your own words

Without notes, explain Timed SQL interview set and explanation in your own words. Start with the purpose: Solve a fresh timed SQL set and explain choices clearly under interview-like constraints. Then name one realistic failure mode and explain how you would prove the result using this check: Save SQL, result controls, time used, confidence, errors and explanation.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Timed SQL interview set and explanation with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: module practical / Gate route after the mastery checklist.